Detecting Fraudulent Bitcoin Transactions

Oct 24, 2026 · @melvin

BT4012 Kaggle competition report · NUS student e1156285 · Code: github.com/melvinloo091/bt4012-bitcoin-fraud-detection

1. Introduction

Bitcoin lets anyone send money without a bank in the middle, which also attracts crime such as scams, ransomware and dark-web markets. The task is to give every transaction a **suspicion score**, so that **illicit** (crime-linked) transactions rank above **licit** ones.

The data. Transactions fall into 49 **time steps**. I learn from steps 1–35 (31,235 labelled transactions, 12% illicit; 110,537 unlabelled) and score 15,329 from steps 36–49. Each has 165 anonymised numbers (1–93 describe the transaction, 94–165 its **neighbours**), and 234,355 "who paid whom" links form a network (a **graph**).

Scoring. Kaggle uses **ROC-AUC**: the chance that a random illicit transaction scores above a random licit one (0.5 = coin toss, 1.0 = perfect). Like an airport security queue, what matters is that the dangerous bags are near the front.

The main difficulty. A large dark market shut down at step 43, inside the test period, after which the illicit share falls from about 11% to under 2%. It is like revising with past papers when the syllabus changes halfway through the exam.

Outcome. Two averaged tree models with recency weighting raised my public score from 0.93654 to **0.95527** (course baseline about 0.92).

2. Methodology

2.1 Data preparation and feature engineering

Missing values. No feature values are missing and the features arrive scaled; only labels are "unknown". Treating unknown rows as licit, **pseudo-labels**, spreading labels to neighbours and an **autoencoder** all lowered local results (wrong guesses teach wrong answers), so the model learns from labelled rows only.

Feature selection. Six "calendar" features (100, 101, 103, 136, 137, 139) track the time step almost exactly (rank correlation above 0.9); like a date printed on each page, they invite rules that say nothing about the future. I removed them from XGBoost only (hardest local

test: 0.915 to 0.921). Dropping the features that shift most before the test period (+0.002, too small to trust), keeping only features 1–93 (−0.010) and **SMOTE** synthetic illicit rows (−0.004) did not help.

Graph-based features. Broad network measures (**node degree**, triangles, clustering, **PageRank**) lowered the practice-exam average (Section 2.2) by about 0.005, and graph neural networks lost on every exam: features 94–165 already summarise neighbours, and two-thirds of links around test transactions lead to missing transactions. Nine simple counts in XGBoost only (links in and out, two-step reach, partners' link counts, share of one-in-one-out "chain" partners) tied locally but raised the public score to 0.95524.

A quirk of the files. The missing transactions are exactly the 46,668 unlabelled ones from steps 36–49, so labelled partners can be counted consistently. Transactions with no labelled partner were illicit 24% of the time (8% otherwise), but the counts only tied (Kaggle: 0.95499), as features 94–165 already carry this signal.

2.2 Model development and validation

Splitting by time. Random cross-validation lets the model learn from the very time steps it is tested on (like practising with the exam paper), so I always split by time; even so, my first model scored 0.995 on steps 29–35 but 0.937 on Kaggle.

Yardstick: 15 "practice exams". Each learns from steps 1 to k and tests on the next 14 steps (k = 7 to 21); the local score is their average ROC-AUC. It put five ordinary models in Kaggle's order, but only a third of a local gain carried over: a compass, not a ruler.

Blind spots and a safety check. The exams cannot test changes aimed at the post-shutdown period, or methods comparing test rows (a "look-alike" model scored 0.9713 locally, 0.87805 on Kaggle), so each candidate's ranking was checked against my best model's before upload.

Hyperparameters and overfitting. Regularisation acts like brakes: too little and the model memorises noise (**overfitting**), too much and it misses small fraud rings. Lighter brakes beat my early heavy ones in 13 of 15 exams; small trees (15 leaves) and a slow learning rate (0.05) guard against overfitting.

Class imbalance. Only 12% of rows are illicit, but ROC-AUC only judges the ordering; weighting illicit rows 7.6 times cost 0.009 locally, so I left the classes as they are.

2.3 Models explored and final strategy

Models explored. Logistic regression (a weighted checklist) was too simple (0.893 on the practice exams); **random forest** and **Extra Trees** (yes/no flowcharts that vote) were weak in hard periods; **neural networks**, including graph ones, lost on every exam. **Gradient**

boosting (XGBoost, LightGBM, CatBoost, HGB), a relay of trees each fixing the last one's mistakes, won every comparison.

Final strategy. The final model has two ingredients:

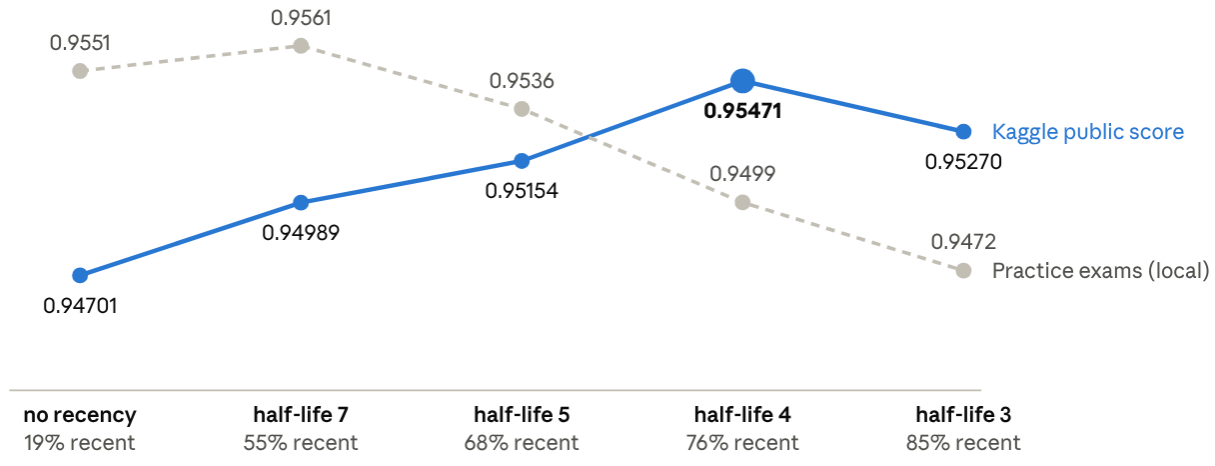
1. **Two models averaged:** XGBoost (300 trees trained with DART, which randomly switches off earlier trees while learning; no calendar features; plus the nine network counts) and LightGBM (150 trees, all 165 features), 50/50; like two doctors, their separate mistakes partly cancel out.
2. **Recency weighting:** each training transaction counts 0.5^((35 − step) / 4), so with this **half-life** of 4, step 31 counts half as much as step 35, like a forecaster trusting last week over last year. Kaggle chose the half-life (Section 3).

The notebook (e1156285.ipynb) regenerates the 0.95527 file exactly. My final selections are my two highest public scores: this model and the same model without DART (0.95524).

3. Results and discussion

Kaggle score peaks at half-life 4, while practice-exam scores fall

ROC-AUC of the two-model average as recency weighting gets stronger (left to right)



Half-life in time steps (smaller = stronger recency). "% recent" = share of training weight on steps 29–35.

Kaggle public leaderboard (11–13 Oct 2026) and the 15 practice exams reproduced in e1156285.ipynb

Recency weighting gave the largest gain: +0.0077 on Kaggle up to half-life 4, then a fall at 3, because stronger recency also throws data away (half-life 4 effectively leaves about 8,800 of 31,235 transactions). The practice exams, which contain no shutdown, moved the other way, and half-life 5 scoring below both neighbours suggests half-life 4 was lucky by about 0.002.

Key submissions. Each adds one change to the row above; the look-alike blend was dropped.

Submission	Change	Practice exams	Kaggle public
Histogram gradient boosting	First boosted trees	0.9310	0.93654
Tuned XGBoost	Small trees, heavy brakes	0.9409	0.94265
Look-alike blend	Near-duplicate score	0.9713	0.87805
Two-model average	Light brakes, + LightGBM	0.9551	0.94701
Recency weighting	Half-life 4	0.9499	0.95471
More trees	300 XGBoost trees	0.9507	0.95484
Network counts	+ 9 network counts	0.9505	0.95524
Final model	**XGBoost with DART**	**0.9498**	**0.95527**

Strengths and limitations. The model is simple, fast and exactly reproducible, and every choice was tested on forward-in-time exams. But the half-life was tuned on the public 30% of rows, where a score can differ from the private one by about ±0.007–0.009 by chance. Most remaining errors are **new fraud rings**: in the hardest exams, 96–98% of missed illicit transactions were linked to other missed ones, so an unseen ring is missed as a whole.

4. Conclusion and reflection

My final model raised the public ROC-AUC from 0.93654 to 0.95527. Three lessons:

1. **A practice test must resemble the real exam:** one easy period promised 0.995, and even 15 forward-in-time exams could not see the shutdown.
2. **Complex is not automatically better:** graph neural networks, autoencoders and the look-alike score lost to plain trees.
3. **Do not over-read small differences:** public gaps below about 0.005 may be luck.

Next, I would look for ways to flag fraud rings unlike any in training.

Acknowledgement of AI assistance. I used two AI assistants. OpenAI Codex (via ChatGPT) set up the repository and notebook and wrote and ran the baselines and early experiments. Anthropic's Claude proposed and coded the practice-exam evaluation and the safety check, wrote and ran most later experiments, built the final notebook and drafted this report. I set the goals, chose which candidates to upload, compared the two assistants' results and chose the final submissions. All experiments used the competition data only.